Lifecycle Testing

Admin Operations and Configuration Manual

Manual 19 of 25
Route: /admin-mock-lifecycle
Group: Tools
Generated: May 10, 2026

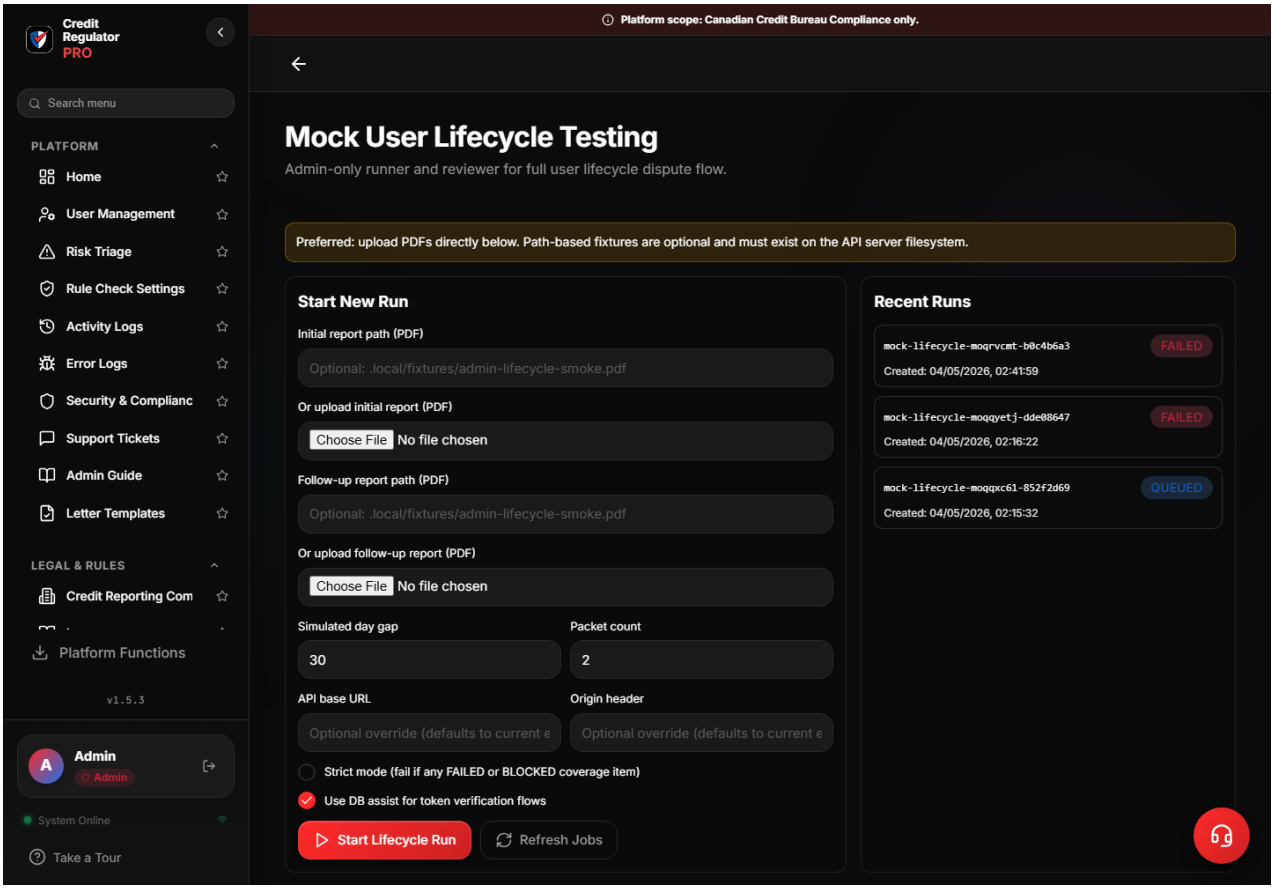
1. Section Overview

Lifecycle Testing runs controlled mock-user workflows for end-to-end validation of registration, upload, parsing, account creation, packet generation, support, and cleanup behavior.

Use this section to validate the platform lifecycle without using a real customer account or production data.

2. Current Admin Screenshot

Screenshot source: localhost development admin environment with test data. Test data is permitted for these manuals and should still be treated as internal.



The screenshot shows lifecycle test controls, run status, workflow stages, and cleanup/result panels.

3. Access and Operating Rules

- Sign in with an account that has admin access. Support users may see some reference sections, but admin configuration and review tools require admin role.
- Use the intended environment. Local admin browser testing uses http://localhost:5175. The backend port, http://localhost:3333, is API-only.
- Treat screenshots, exports, logs, report text, and record IDs as internal test-environment material. Do not paste them into public channels.
- Confirm the page route, active filters, selected user, selected report, and selected environment before any save, delete, reset, dismissal, or finalize action.
- When an action affects customer-facing findings, packets, pricing, feature flags, or parser truth, record the reason and verify the downstream record after refresh.

- Do not run lifecycle cleanup against real users.
- Use the local or staging admin UI flow for operations review rather than server-local file paths.
- Failed lifecycle stages should be traced through source pages and logs before rerunning repeatedly.

4. Screen Anatomy and Controls

Area	What it means	Admin procedure
Run controls	Create and execute mock lifecycle tests.	Use disposable mock accounts only.
Stage output	Shows each lifecycle stage and result.	Use failed stage context to decide whether to open Error Logs or source pages.
Cleanup controls	Remove mock data when approved.	Confirm target is a mock user before cleanup.
Result summary	Shows completion, coverage, and residual data state.	Use as validation evidence after checks.

5. Step-by-Step Instructions

Run a mock user lifecycle

Use before releases or after changes to auth, upload, parser, packet, support, or cleanup behavior.

Procedure

1. Open Lifecycle Testing from the Tools group.
2. Confirm the environment and that the configured account/email is disposable mock data.
3. Start the lifecycle run and monitor each stage.
4. When a stage fails, stop and read the failure details before rerunning.
5. Open Error Logs for server/API failures and Activity Logs for successfully completed mutations.
6. After a successful run, review generated artifacts, account records, and packet status if the test covers them.
7. Run cleanup only after confirming the test data is no longer needed.

Decision options

- Every required stage passed or has a documented expected failure.
- Cleanup removed mock data without affecting real users.

Verification

- Run: use for end-to-end validation.
- Inspect failure: use before rerun when any stage fails.
- Cleanup: use after validation with mock user confirmation.

Validate upload-to-packet flow

Use when parser, packet, delivery, or template changes may affect the main customer lifecycle.

Procedure

1. Start a lifecycle run with a representative mock report.
2. Confirm upload completes and parser output is created.
3. Inspect tradeline and finding creation stages.
4. Confirm packet generation uses correct user profile, template, bureau/creditor references, and evidence.
5. Open related source pages for any failed or ambiguous stage.
6. Record run ID, mock user email, report artifact ID, and packet ID for release notes.

Verification

- Upload, parser, tradeline, finding, packet, and cleanup stages are verified.

6. Common Tasks With Drilldown Examples

Investigate a failed lifecycle stage

Example: Packet generation fails after parser output succeeds.

Drilldown procedure

1. Read the failed stage name and message.
2. Open Error Logs around the failure time and search by route or endpoint.
3. Open Activity Logs to identify which earlier stages succeeded.
4. Open the source page for the failed artifact, account, or packet.
5. Decide whether the failure is test data, parser output, template, permissions, or application defect.
6. Escalate with run ID, stage, source IDs, and screenshot.

Best practices for this task

- Do not rerun blindly if the failure created partial data.
- Preserve source IDs before cleanup.

Done when

- Failure cause is categorized and escalation is reproducible.

Clean up mock lifecycle data

Example: A successful test run has served its validation purpose.

Drilldown procedure

1. Confirm the account is a mock/disposable test user.
2. Record run ID and key artifacts before cleanup if needed for release evidence.
3. Run cleanup from the lifecycle tool.
4. Review cleanup summary for remaining records.
5. Open User Management or logs if residual records remain unexpectedly.

Best practices for this task

- Never use cleanup for real customer repair.
- Use User Management reset workflow for approved real-user repair cases.

Done when

- Mock data is removed or residuals are documented.

7. Configuration and Change-Control Scope

- Lifecycle tests create, mutate, and clean up mock data.
- They validate workflows but should not be used with production customer records.
- Cleanup behavior can remove mock reports, tradelines, packets, sessions, and related rows.

8. Common Errors

Problem	Likely cause	Admin response
Access denied or redirected to login	The session expired, the role is not admin, or the wrong environment is open.	Sign back in, confirm admin role, and reopen the route from the sidebar instead of using an old bookmark.
Expected record is missing	Search terms, tabs, filters, pagination, or environment selection are hiding the record.	Clear filters, search by exact ID or email, verify the environment, and compare Activity Logs when available.
Save appears to do nothing	Required fields are missing, there are no dirty changes, or the API returned a validation error.	Read the inline validation text, correct one field at a time, save again, refresh, and check Error Logs if the state still does not persist.
lifecycle testing state conflicts with another page	The source record was changed elsewhere or the current page is stale.	Refresh the page, reopen the source record, and use timestamps or record IDs to decide which state is authoritative.

9. Troubleshooting

Check	Why it matters	How to proceed

Check	Why it matters	How to proceed
Start with a refresh	A refresh confirms whether the lifecycle testing view is stale or whether the backend rejected the action.	Refresh once, then repeat the exact search or selection. Do not repeat destructive actions until the latest state is visible.
Check Activity Logs	Successful admin actions should leave an audit trail for sensitive workflows.	Search by actor, affected user, entity ID, route, or timestamp. Use the log entry as confirmation, not just the toast message.
Check Error Logs	Repeated UI failures usually have matching server or API errors.	Search around the action time. Capture the route, status, error summary, and request context for developer follow-up.
Escalate with evidence	Developer review is faster when the exact record and reproduction path are known.	Provide environment, route, selected filters, record ID, expected outcome, actual outcome, screenshot, and timestamp.

10. Best Practices

- Use disposable mock accounts only.
- Record run IDs and artifact IDs.
- Investigate failed stages before reruns.
- Confirm cleanup target before deleting mock data.

11. Completion Checklist

- The lifecycle testing page was reopened or refreshed after the change.
- The expected saved state remained visible after refresh.
- Related logs, downstream records, or generated artifacts agree with the visible page state.
- Any customer-facing wording, finding status, packet state, or support note was reviewed for factual and neutral language.
- Any unresolved risk, failed test, suspicious log, or unclear source evidence was escalated with record IDs and screenshots.

12. Related Admin Areas

- User Management
- Parser Testing
- Error Logs
- Activity Logs
- Support Tickets